



ВТУ «Св. Св. Кирил и Методий»

Факултет «Математика и информатика»

Курсова работа

Стандартът AES

Разработил:

Николета Панайотова

Спец. “Информационни ситеми”

Фак. №: МПИ-10679р

Проверил:

Доц. Д-р В. Бакоев

Доц. З. Върбанов

Велико Търново

2012г.

1. История:

През 1997 г., Американският НИСТ (Национален Институт за Стандарти и Технологии) призовава да се намери заместител на DES. Бяха проучени 15 алгоритъма, и въз основа на различните критерии, към края на 2000 г., Белгийския алгоритъм Rijndael (произнася се Райндал), предложен от *Joan Daemen* и *Vincent Rijmen*, бе избран. Всеки модел бе тестван на многократно и на няколко нива и Rijndael не беше на първо място при всички тестове, но доказва голяма стабилност при всеки различен тест и запазваше добри постижения. Той бе избран заради неговите способности за адаптиране и неговите постижения. Заради произхода си, AES бе наречен стандарт. Той е свободен за използване, без ограничение на използваемост или патент.

2. Принцип:

AES работи с правоъгълни блокове от четири линии и N_C колони и всеки термин $x_{i,j}$ (наречен байт) се състои от 8 бита ($b=b_7b_6b_5b_4b_3b_2b_1b_0$) и може да бъде представен в алгебричен вид под формата на полином от степен ≤ 7 ($b = b_7X^7 + b_6X^6 + b_5X^5 + b_4X^4 + b_3X^3 + b_2X^2 + b_1X + b_0$) с коефициенти в F_2 (0 или 1). Ключът може да бъде с дължина от 128, 156 или 256 бита, същото се отнася за съобщението и за кодираното съобщение.

3. Описание:

Съобщението и ключът се разделят на байтове и се поставят в таблици. Съобщението съдържа $t_d = 16$ байта $p_0, p_1, \dots, p_{15}$, които са наредени в една таблица от 4 линии и 4 колони. Таблицата се запълва колона по колона. По същият начин ключът е разделен на байтове ($t_k = 16$, $t_k = 24$ или $t_k = 32$ байта) $k_0, k_1, \dots, k_{t_k-1}$. Тези байтове са също поставени в една таблица от 4 линии и N_k колони ($N_k=4$, $N_k=6$, $N_k=8$)

Таблицата с **бров** завъртания е следната:

p_0	p_4	p_8	p_{12}	k_0	k_4	k_8	k_{12}	k_{16}	k_{20}	k_{24}	k_{28}
p_1	p_5	p_9	p_{13}	k_1	k_5	k_9	k_{13}	k_{17}	k_{21}	k_{25}	k_{29}
p_2	p_6	p_{10}	p_{14}	k_2	k_6	k_{10}	k_{14}	k_{18}	k_{22}	k_{26}	k_{30}
p_3	p_7	p_{11}	p_{15}	k_3	k_7	k_{11}	k_{15}	k_{19}	k_{23}	k_{27}	k_{31}

Фиг 3.1. Съобщението и ключът (в случай на $N_k=8$)

В зависимост от дължината на ключа, броя на завъртанията или трансформациите се нарича n_r . броя може да се види в следната таблица:

N_k	4	6	8
n_r	10	12	14

Започвайки от първоначалният ключ, системата създава $n_r + 1$ ключове за завъртане и всеки от тях съдържа 16 байта. Тези ключове ще бъдат записани в една едномерна таблица ТК и са записани по следният начин: ТК[0], ТК[1], ..., ТК[n_r]

W следващите стъпки ще се разгледа начин, по който се изчисляват тези ключове спрямо ключа К.

В общ вид системата работи по следният начин: тя взима като вход една таблица от данни St (некриптиран текст), който е променен чрез процедурата и изпратен на изхода (криптиран текст).

Бход: таблицата St и ключът К

Изход: таблицата St модифицирана

AES(St, K)

BEGIN

KeyExpansion(K, ТК);

AddRoundKey(St, ТК[0]);

for(i=1; i < n_r ; i++)

 Round(St, ТК[i]);

FinalRound(St, ТК[n_r]);

END

Процедурите *Round* и *FinalRound* са композирани в самите себе си.

Бход: таблицата St и ключът на завъртане Т

Изход: таблицата St модифицирана

Round(St, Т)

BEGIN

SubBytes(St);

ShiftRows(St);

MixColumns(St);

AddRoundKey(St, Т);

END

```

FinalRound (St,T)
BEGIN
SubBytes(St);
ShiftRows(St);
AddRoundKey(St,T);
END

```

4. Алгоритъма:

a) SubBytes: тази процедура е единствената трансформация която не е линейна. От нея идва силата на системата. Тя изпозва операции върху полето от 256 елемента.

- **Поле от 256 елемента:**

Нека полиномът $P(X) = X^8 + X^4 + X^3 + X + 1$ е полином с коефициенти в $F_2 = \{0,1\}$
 Елементите от полето с 256 елемента ще бъдат байтовете $b_7b_6b_5b_4b_3b_2b_1b_0$
 представени като полиноми $b(X) = b_7X^7 + b_6X^6 + b_5X^5 + b_4X^4 + b_3X^3 + b_2X^2 + b_1X + b_0$
 което ни позволява да дефинираме следните две операции:

Събиране $a_7a_6a_5a_4a_3a_2a_1a_0 + b_7b_6b_5b_4b_3b_2b_1b_0 = c_7c_6c_5c_4c_3c_2c_1c_0$ със $c(X) = a(X) + b(X)$
 което ни дава $c_i = a_i \oplus b_i$

Умножаване $a_7a_6a_5a_4a_3a_2a_1a_0 \times b_7b_6b_5b_4b_3b_2b_1b_0 = c_7c_6c_5c_4c_3c_2c_1c_0$
 със $c(X) = a(X) \times b(X) \bmod P(X)$

И по този начин има една структура на поле и тогава всеки ненулев елемент е обратим. Ще наречем $g: F_{256} \rightarrow F_{256}$ дефинира с

$$g(x) = \begin{cases} 0 & \text{ако } x=0 \\ x^{-1} & \text{ако } x \neq 0 \end{cases}$$

- **Функцията ϕ :** дефинирано е $b = \phi(a)$ с помощта на следната матрица

$$\begin{pmatrix} b_7 \\ b_6 \\ b_5 \\ b_4 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a_7 \\ a_6 \\ a_5 \\ a_4 \\ a_3 \\ a_2 \\ a_1 \\ a_0 \end{pmatrix} \oplus \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

$$b(x) = ((X^4 + X^3 + X^2 + X + 1) \times a(x) \bmod (X^8 + 1)) + (X^6 + X^5 + X + 1)$$

Забелязваме, че $g^{-1} = g$ и че ϕ^{-1} е дефинирана като:

$$\begin{pmatrix} b_7 \\ b_6 \\ b_5 \\ b_4 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} a_7 \\ a_6 \\ a_5 \\ a_4 \\ a_3 \\ a_2 \\ a_1 \\ a_0 \end{pmatrix} \oplus \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

За процедурата *SubBytes* дефинираме тогава: $s(a) = \phi(g(a))$

Също така имаме: $s^{-1}(b) = g(\phi^{-1}(b))$

Процедурата *SubBytes* изпълнява s на всеки байт от входа St .

a ₀	a ₄	a ₈	a ₁₂		b ₀	b ₄	b ₈	b ₁₂
a ₁	a ₅	a ₉	a ₁₃	→	b ₁	b ₅	b ₉	b ₁₃
a ₂	a ₆	a ₁₀	a ₁₄		b ₂	b ₆	b ₁₀	b ₁₄
a ₃	a ₇	a ₁₁	a ₁₅		b ₃	b ₇	b ₁₁	b ₁₅

Фиг 4.1. Трансформацията *SubBytes*

a) ShiftRows

Процедурата извършва едно преместване в ляво върху всяка линия от входната таблица. Броя на полетата, от който декларираме линията i ($0 \leq i \leq 3$) е i .

Обратната трансформация се изчислява лесно.

a ₀	a ₄	a ₈	a ₁₂		a ₀	a ₄	a ₈	a ₁₂
a ₁	a ₅	a ₉	a ₁₃	→	a ₅	a ₉	a ₁₃	a ₁
a ₂	a ₆	a ₁₀	a ₁₄		a ₁₀	a ₁₄	a ₂	a ₆
a ₃	a ₇	a ₁₁	a ₁₅		a ₁₅	a ₃	a ₇	a ₁₁

Фиг 4.2. Трансформацията *ShiftRows*

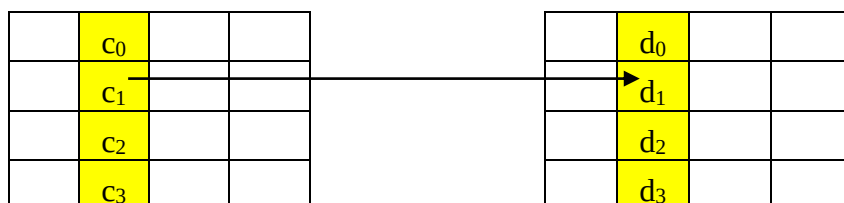
b) MixColumns:

Трансформацията *MixColumns* се състои от изпълнението на една и съща

трансформация на всяка колона от таблицата с данни по следния начин:

нека колоната $c = (c_1, c_2, c_3, c_4)^t$

Елементите c_i са елементи от F_{256} . Всяка колона c е трансформирана в колона d с помощта на линейната трансформация издадена от матрицата, чиито коефициенти принадлежат на F_{25} , който байтове се записват в шестнадесетична бройна система.



Фиг 4.2. Трансформацията *MixColumns*

$$\begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \times \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix}$$

Трансформацията се състои от умножение по полином:

$$A(X) = 03.X^3 + 01.X^2 + 01.X + 02$$

по модул $1+X^4$:

$$d(X) = A(X) \times c(X) \mod X^4+1$$

където

$$c(X) = c_0 + c_1X + c_2X^2 + c_3X^3$$

и

$$d(X) = d_0 + d_1X + d_2X^2 + d_3X^3$$

Полиномът $A(X)$ е обратим по модул $1+X^4$ и обратният му полином е

$$B(X) = 0B.X^3 + 0D.X^2 + 09.X + 0E$$

И по този начин можем да намерим $c(X)$ от $d(X)$ като се изпълни произведението

$$c(X) = B(X) \times d(X) \mod X^4+1$$

или чрез матрицата

$$\begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix} \times \begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{pmatrix}$$

c) AddRoundKey

Процедурата *AddRoundKey* е много елементарна, тя се състои от извършването на

операцията OR между 128-те бита на състоянието St и 128-те бита на ключа за завъртане T .

Получаваме една нова стойност на състояние: $St := St \oplus T$

d) KeyExpansion

Ключът за кодиране K , който е запаметен в една таблица от 4 линии и N_k на брой колони ($N_k = 4, 6, 8$) е одължена в една таблица W съдържаща 4 линии и $4 \cdot n_r + 1$ колони. Ключът за завъртане $TK[i]$ ($0 \leq i \leq n_r$) е даден от четирите колони $4 \cdot i$, $4 \cdot i + 1$, $4 \cdot i + 2$, $4 \cdot i + 3$ от таблица W .

Има 2 начин за създаване на таблицата W в зависимост дали $N_k = 4, 6$ или $N_k = 8$. Процедурата за създаване се нарича *ExpandedKey*.

❖ В случай на $N_k = 4$ или $N_k = 6$

Вход: ключът K (във формата на таблица)

Изход: таблицата W

$ExpandedKey(K, W)$

BEGIN

for($j=0$; $j < N_k$; $j++$)

for($i=0$; $i < 4$; $i++$)

$W[i, j] = K[i, j];$

for($j=0$; $j < N_k$; $j++$) {

if ($j \bmod N_k == 0$) {

$W[0, j] = W[0, j - N_k] \oplus s(W[1, j-1]) \oplus RC[j / N_k];$

for($i=1$; $i < 4$; $i++$)

$W[i, j] = W[i, j - N_k] \oplus s(W[i+1 \bmod 4, j-1]);$

}

else {

for($i=1$; $i < 4$; $i++$)

$W[i, j] = W[i, j - N_k] \oplus s(W[i, j-1]);$

}

}

END

Процедурата използва функцията s върху байтовете. Тя използва също константите от F_{256} дадени чрез: $RC[i] = \alpha^i$ където α е елементът от F_{256} , който съответства на полинома X ($\alpha=02$). Вдигане на степен i става в полето F_{256} .

❖ В случей на $N_k = 8$

Бход: ключът К (във формата на таблица)

Изход: таблицата W

ExpandedKey(K,W)

BEGIN

for(j=0; j<N_k; j++)

for(i=0; i<4; i++)

W[i,j] = K[i,j];

for(j=N_k; j<4(n_r+1); j++) {

if (j mod N_k == 0) {

W[0,j] = W[0,j - N_k] ⊕ s(W[1, j-1]) ⊕ RC[j/ N_k];

for(i=1; i<4; i++)

W[i,j] = W[i,j-N_k] ⊕ s(W[i+1 mod 4, j-1]);

}

else if(j mod N_k == 4) {

for(i=0; i<4; i++)

W[i,j] = W[i,j-N_k] ⊕ s(W[i, j-1]);

}

else {

for(i=0; i<4; i++)

W[i,j] = W[i,j-N_k] ⊕ W[i, j-1];

}

}

END